Machine Learning (2026) 115:245
https://doi.org/10.1007/s10994-026-07117-8

MMT: Achieving Exact Federated Unlearning with Improved Post-unlearning Performance

Ze Yu Zhang[1,2] · Nhung Bui[1] · Arun Verma[3] · Bolin Ding[2] · Bryan Kian Hsiang Low[1,3]

Received: 31 October 2025 / Revised: 1 July 2026 / Accepted: 2 July 2026
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract

Federated learning allows multiple clients to collaboratively train a shared global model by sharing only model updates, such as gradients or parameters, to a central server without sharing their private data. Even though the clients do not share their private data, their shared model updates across multiple communication rounds indirectly influence the global model. This indirect influence poses significant challenges in many real-life applications, where it is necessary to completely remove a specific client's influence (exact unlearning) from the global model. For example, exact unlearning may be required to comply with privacy regulations such as the GDPR's Right to be Forgotten, or to remove the influence of malicious or poisoned clients in security-sensitive applications (e.g., jointly trained fraud detection models), where eliminating their influence is necessary to maintain the model's performance. While retraining the global model with the remaining clients from scratch can achieve exact unlearning, it inevitably introduces significant deployment delays and leads to severe performance degradation due to random initialization, making it impractical for time-sensitive applications. To overcome these limitations, this paper proposes two exact federated unlearning algorithms, Bi-Models Training (BMT) and Multi-Models Training (MMT), that ensure immediate exact unlearning by design while achieving better performance post-unlearning. BMT preserves isolated local model copies during training and uses them to provide a stronger initialization for the global FL model after unlearning. To capture the joint influence of multiple clients, MMT trains a carefully constructed set of sub-FL models on disjoint client subsets to avoid overlapping influence, and then aggregates the most influential unaffected models to reinitialize the global FL model. Our experiments on vision and language tasks demonstrate that BMT consistently outperforms existing exact unlearning baselines, while MMT achieves over 2× improvement in post-unlearning performance, all while maintaining exact unlearning guarantees and compatibility with arbitrary federated learning protocols.

Keywords Federated learning · Exact federated unlearning · Improved post-unlearning performance · Multi-models training

Ze Yu Zhang, Nhung Bui, and Arun Verma contributed equally to this work.

Editors: Zahraa S. Abdallah, Annalisa Appice, Przemyslaw Biecek, Andrea Tagarelli.

Extended author information available on the last page of the article

Published online: 12 October 2026

1 Introduction

Deviating from the assumption of centralized data availability, real-world data is often distributed across individuals and organizations. A machine learning model trained in isolation often fails to generalize beyond its local data distribution (Li et al., 2020; Zhao et al., 2018). In contrast, leading production systems, such as web search auto-completion (Shokouhi, 2013), Facebook ad ranking (He et al., 2014), and Instagram recommendations (Vorotilov and Shugaepov, 2023), use millions of diverse user interactions to achieve state-of-the-art accuracy and robustness.

As data privacy regulations and policies become increasingly stringent (CCPA, 2018; GDPR, 2016), this raises questions about whether collecting and storing large, diverse datasets remains sustainable for training good machine learning (ML) models. The data decentralization and the need to uphold data privacy in ML have driven the development of *federated learning* (FL) (Kairouz et al., 2021; McMahan et al., 2017), a collaborative learning paradigm that allows participating clients (data owners) to train local models on their private data and share only model updates with a trusted server for aggregation into a global model, thereby avoiding the need to share private data. FL remains highly relevant across many privacy-sensitive domains, such as healthcare (Brisimi et al., 2018; Rieke et al., 2020), finance (Byrd and Polychroniadou, 2020; Long et al., 2020), and edge computing (Hard et al., 2018; Lim et al., 2020), where the need to protect sensitive data makes traditional ML methods that require centralized data access impractical. During the FL process, potential new clients may request to join the FL training; likewise, existing clients may request to opt out of the process. When a client leaves, privacy regulations such as the GDPR's ***Right to be Forgotten*** (GDPR, 2016) allow them to have their data and its lineage deleted from the global model. A similar need may also arise from the FL server side, e.g., a server wants to ***remove a malicious client*** whose model updates sabotage the global model performance and/or pose serious privacy or security risks (Fang et al., 2020; Tolpegin et al., 2020). While motives differ, both problems require the global model to ***unlearn*** the contributions of a specific client, a process known as *federated unlearning* (FU) (Liu et al., 2021, 2024; Wang et al., 2022).

Prior works have shown that FL models can memorize private training data from clients (Geiping et al., 2020; Nasr et al., 2019; Zhu et al., 2019). Due to multiple communication rounds of the FL training, it becomes impossible to completely remove a client's data influence from the trained global model. Therefore, the most straightforward method to achieve the exact FU is to restart the federated learning process from scratch with the remaining clients, i.e., retraining from scratch (RFS), in order to recover the global model's pre-unlearning performance (Bourtoule et al., 2021; Liu et al., 2024). Although RFS is simple, restarting FL from random initialization can significantly degrade global model accuracy in the early rounds after unlearning compared to its pre-unlearning performance. Such degradation is unacceptable in critical, time-sensitive applications such as healthcare (Dhade and Shirke, 2023; Prayitno et al., 2021), finance (Long et al., 2020), and supply chains (Wallace et al., 2003), where uninterrupted service is essential, and delays can incur severe consequences (Liu and Ren, 2025; Rostek et al., 2022; Wallace et al., 2003). Moreover, RFS may require many FL training rounds to recover desirable performance, forcing a trade-off between delayed deployment of an unlearned model and continued use of the pre-unlearning model, which risks privacy violations and suboptimal performance in the presence of malicious

clients (Fang et al., 2020; Jin et al., 2021; Rashid et al., 2023; Shi et al., 2022). This problem naturally raises the following question: ***How to achieve exact FU with minimal delay while ensuring better post-unlearning performance?***

The works most closely related to ours include (Qiu et al., 2023; Tao et al., 2024; Xiong et al., 2023), but their algorithms either *degrade the global model performance* with large quantization (Xiong et al., 2023) or cluster sensitivity (Qiu et al., 2023), or *require significant computational resources* to store intermediate checkpoints (Tao et al., 2024), leading to slower post-unlearning recovery and causing undesirable disruptions and losses for delayed deployment of an unlearned global model in time-sensitive services. To address these challenges, we propose novel exact FU algorithms, Bi-Models Training (BMT) and Multi-Models Training (MMT), which ***achieve exact FU without any delay while having better post-unlearning performance*** than RFS, while can be easily integrated with any FL protocol.

To achieve better post-unlearning performance, BMT preserves ***isolated copies of local models*** and reuses clients' existing knowledge residing in these models during unlearning ***to get better initialization***. Despite being computationally efficient, these local models fail to capture the joint influence of multiple clients on the global model. Although training models for every subset of clients, referred to as *sub-FL models* (see Definition 1), can theoretically capture all possible client influences, this naive approach is computationally prohibitive due to the exponential number of sub-FL models. Furthermore, it may also introduce the problem of ***overlapping influence*** (see Definition 2), where a single client impacts multiple sub-FL models. As a result, we propose MMT that only trains each sub-FL model on disjoint subsets of clients to avoid overlapping influence and upon unlearning aggregates the most influential unaffected sub-FL models, as determined by the Influence Degradation Score (see Definition 3), to achieve an improved initialization of the global model. We empirically validate the effectiveness of both BMT and MMT across both vision and language tasks (see Sect. 5). Specifically, our key contributions are as follows:

- We propose two exact FU algorithms, BMT and MMT, that are computationally more efficient than existing algorithms and achieve better post-unlearning performance. Both algorithms are readily compatible with any FL protocol.
- To capture joint influence, MMT constructs an ***influence tree*** composed of local and sub-FL models that explicitly eliminates overlapping influence by design. We further prove that a binary influence tree constructed using Huffman coding maximizes the overall joint influence of clients on the FL re-initialization model among all possible k-ary influence tree structures (Theorem 1).
- Extensive experiments on vision and language tasks show that BMT outperforms existing baselines, while MMT achieves over 2× the performance of state-of-the-art exact FU algorithms, all while maintaining exact FU guarantees.

2 Related Works

Machine Unlearning. Machine unlearning (MU) studies how to remove the influence of selected training data from a trained model without fully retraining it from scratch. Most existing MU methods are developed in *centralized* settings and are therefore complemen-

tary to, but not directly comparable with, client-level federated unlearning. Representative examples include Amnesiac machine learning (Graves et al., 2021), which studies efficient data deletion for centrally trained models, and SalUn (Fan et al., 2024), which proposes gradient-based weight saliency to localize forgetting updates. In contrast, our work focuses on *exact, client-level* unlearning in cross-silo FL, where the unlearned client's data is not assumed to be accessible during the unlearning process.

Federated Unlearning. Existing works on federated unlearning (FU) (Liu et al., 2021, 2024; Wang et al., 2022) can be broadly categorized into exact FU and approximate FU. Exact FU approaches (Qiu et al., 2023; Tao et al., 2024; Xiong et al., 2023) provide a guarantee for the complete removal of the unlearning clients from the global model, which is typically verifiable through their algorithmic designs. In contrast, approximate FU approaches (Cao et al., 2023; Halimi et al., 2022; Liu et al., 2021; Pan et al., 2025) aim to efficiently approximate retraining and require rigorous empirical evaluation to quantify the residual influence of the unlearning clients but lack an unlearning guarantee. Notable approximate FU mechanisms include reversing the gradient of the unlearning clients (Pan et al., 2025; Wu et al., 2022), calibrating the updates from the remaining clients (Liu et al., 2021; Wu et al., 2022), training-time augmentation with auxiliary unlearning components (e.g., FedAU) (Gu et al., 2024), and parameter-perturbation methods such as weight negation (e.g., NoT) (Khalil et al., 2025). While being more predominant, previous works note that existing approximate FU algorithms can leave behind residual influence of the unlearning clients and compromise their privacy (Lamri et al., 2026; Wang et al., 2024). Furthermore, the lack of effective and standardized unlearning metrics to quantify this influence raises the barrier to designing reliable approximate FU algorithms (Romandini et al., 2025). As real-world deployments are expected to strictly adhere to the "Right to be Forgotten", there is an urgent need for further exploration into exact unlearning that provides strong guarantees for the removal of unlearning clients.

Exact Federated Unlearning. FU algorithms with exact unlearning guarantees are under-explored (Qiu et al., 2023; Tao et al., 2024; Xiong et al., 2023). For example, ExactFun (Xiong et al., 2023) and FATS (Tao et al., 2024) use quantization and sampling, respectively, to efficiently restart FL training from a stored checkpoint where the influence of the leaving client is negligible. FedCIO (Qiu et al., 2023) adopts a client-clustering algorithm based on data distribution and restricts FL retraining to only a model subset trained on the unlearned client data. However, these algorithms require significant resources to store intermediate checkpoints, and they can degrade global model performance due to quantization or sensitivity to clustering. Moreover, in the worst case, these algorithms are slow in FL retraining, leading to undesirable delays in deploying the unlearned global model. Our proposed algorithms, BMT and MMT, address these limitations.

3 Problem Setting

3.1 Federated Learning

We consider a **centralized cross-silo** FL setting, where a central, trusted server coordinates training across distributed clients. We assume that a common hypothesis class $\mathcal{H}$ is shared by the server and the clients. The server owns a global model $h_\theta \in \mathcal{H}$, with parameters

$\theta \in \Theta$. In each communication round (*round* for short), the server iteratively updates the global model using the shared model updates from a set of clients $\mathcal{C} := \{1, ..., N\}$, with N clients. Without loss of generality, client c possesses a local, supervised dataset $\mathcal{D}_c \subset \mathcal{X} \times \mathcal{Y}$ defined over the input space $\mathcal{X}$ and the label space $\mathcal{Y}$, which is not shared with the central server or other clients $c' \neq c, c' \in \mathcal{C}$. We denote by $\mathcal{D} := \{\mathcal{D}_c\}_{c=1}^N$ a set of local datasets.

An FL training algorithm $\mathcal{A} : \mathcal{C} \times \mathcal{D} \to \Theta$ generates global model parameters $\theta_t \in \Theta$. One of the popular goals for $\mathcal{A}$ is minimizing the empirical risk/loss across $\mathcal{C}$, i.e.,

$$\arg\min_{\theta \in \Theta} \sum_{c=1}^{N} w_c f(\mathcal{D}_c; \theta), \tag{1}$$

where $f(\mathcal{D}_c; \theta) := \frac{1}{|\mathcal{D}_c|} \sum_{(x,y) \in \mathcal{D}_c} \ell(h_\theta(x); y)$ is the empirical loss measured on the local dataset of client c, with the loss function ℓ, and weighted proportionally to its cardinality $w_c := \frac{|\mathcal{D}_c|}{\sum_{c'=1}^{N} |\mathcal{D}_{c'}|}$.

3.2 Standard FL Protocol

We denote the global model parameters at the t-th round by $\theta^{(t)}$. The server broadcasts $\theta^{(t)}$ (with $\theta^{(0)}$ often being randomly initialized) to all clients. Then, client c updates the copy of $\theta^{(t)}$ via empirical risk minimization on their local dataset $\mathcal{D}_c$ and sends the model update $\Delta\theta_c^{(t)}$ back to the server. We refer to this client-level training process as *local training*. The server aggregates the updates from all clients in $\mathcal{C}$ and updates the global model, e.g., if $\Delta\theta_c^{(t)}$ is the parameter difference resp. to $\theta^{(t)}$, FedAvg uses a weighted average aggregation method, i.e., $\theta^{(t+1)} = \theta^{(t)} + \sum_{c=1}^{N} w_c \Delta\theta_c^{(t)}$. The server repeats the process until $h_{\theta^{(t+1)}}$ converges or a stopping criterion (e.g., maximal T rounds) is met. The parameters of a fully trained global model at the end of the FL training process are denoted by θ^*.

3.3 Exact Federated Unlearning

Let $\mathcal{I}(h_\theta) \subseteq \mathcal{C}$ denote the subset of clients whose updates or data have influenced the model h_θ through training or FL aggregation. We say that the global FL model h_θ has *exactly unlearned* a client c if $c \notin \mathcal{I}(h_\theta)$. Therefore, exact federated unlearning refers to the process of completely removing the influence of a client's training data from the global model.

4 Exact Federated Unlearning via MMT

We next describe our novel algorithm, MMT, for ***achieving exact federated unlearning*** that completely removes the client's influence while giving better post-unlearning performance and shorter delay than RFS and other existing algorithms.

4.1 Multi-Models Training (MMT)

We use the term *global model* to refer to the FL model trained on all clients' data and used for deployment. To have a better global model after unlearning, we must design a new FL training process that allows exact federated unlearning while having a better initialization than random weights. One way to achieve better initialization is to design algorithms that can exploit the remaining clients' existing knowledge. To do this, we propose an algorithm named Multi-Models Training (MMT) that can be incorporated into any existing federated learning framework. The main idea of MMT is to have an additional model for each client that is only trained on its data, making these models unaffected by other clients' training data. We refer to this model as *local model*. However, the local model only accounts for an individual client's influence and does not capture the joint influence of multiple clients. Since all clients influence the global model, we should also capture the joint influence of different clients and then use it to better reinitialize the global model. We can train FL models to capture the joint influence using only a subset of clients. We refer to such FL models as ***sub-FL models*** and next define them formally.

Definition 1 (*Sub-FL Model*) Given a subset of clients $S \subset C$ with $|S| \geq 2$, a sub-FL model is trained from scratch by the central server using the same FL protocol as the global model to minimize the empirical risk over all clients in S.

One can train all possible sub-FL models (power set of clients excluding the global model, singletons (local models), and the empty set) to capture the influence of all possible interactions among clients. However, it is not computationally feasible as these sub-FL models increase exponentially with the number of clients (i.e., $2^n - n - 2$ for n clients). Another problem of training arbitrary sub-FL models can lead to a situation of ***overlapping influence***, defined as follows:

Definition 2 (*Overlapping Influence*) Let h_{θ_i} and h_{θ_j} be two sub-FL models with influence sets $\mathcal{I}(h_{\theta_i}), \mathcal{I}(h_{\theta_j}) \subseteq \mathcal{C}$. They exhibit *overlapping influence* if $\mathcal{I}(h_{\theta_i}) \cap \mathcal{I}(h_{\theta_j}) \neq \emptyset$ and neither set contains the other, i.e., $\mathcal{I}(h_{\theta_i}) \not\subseteq \mathcal{I}(h_{\theta_j})$ and $\mathcal{I}(h_{\theta_j}) \not\subseteq \mathcal{I}(h_{\theta_i})$.

When two sub-FL models share a client, that client's loss is effectively counted twice in the aggregation. Existing works show that such uneven re-weighting shifts the solution away from the (unweighted) empirical risk minimizer and can strictly increase expected error (Alain et al., 2015; Bottou et al., 2018; Byrd & Lipton, 2019; Cortes et al., 2010; Shalev-Shwartz & Ben-David, 2014). Empirically, duplicating examples or clients is known to hurt generalization in both vision and code-model settings (Aghabagherloo et al., 2025; Allamanis, 2019; Buda et al., 2018). To avoid this, sub-FL models should be trained on disjoint client subsets or trained such that one sub-FL model's client set is a proper superset of another sub-FL model's client set.

One possible way to achieve this is to organize sub-FL models in a hierarchical tree structure. In the tree structure, the root node represents the global model, leaf nodes correspond to local models, and intermediate nodes represent sub-FL models, with each child node having a disjoint set of clients compared to its siblings. As we move from the root

node to the leaf nodes, each sub-FL model branches into further subsets, maintaining either disjoint relationships or superset relations.

The ***tree structure ensures varying levels of influence among sub-FL models*** throughout the hierarchy. We refer to this hierarchical tree structure as an ***influence tree***. After unlearning a client, we should aggregate the sub-FL models with greater influence (those influenced by more remaining clients) and the local models to get the initialization for the global model. If the number of models to aggregate is less, it implies that the initialization of the global model contains more joint influence of remaining clients. This relationship inspires our metric ***influence degradation score***, which quantifies the quality of an influence tree and is defined as follows.

Definition 3 (*Influence Degradation Score* (*IDS*)) Let $\mathcal{T}$ be any influence tree structure. The influence degradation score for $\mathcal{T}$, denoted by $s(\mathcal{T})$, is the average number of sub-FL and local models that are aggregated to get the initial global model after unlearning any client.

While the tree structure inherently avoids overlapping influence, it remains unclear which structure minimizes IDS, as unlearning probabilities, i.e., the likelihood of clients requesting unlearning may vary across clients. Since our goal is to minimize IDS, we show that a binary influence tree constructed via Huffman coding achieves the lowest IDS among all k-ary influence tree structures for $k > 2$.

Theorem 1 *Given a k-ary influence tree $\mathcal{T}$ with $k > 2$, there exists a binary influence tree $\mathcal{T}_2$ that has smaller IDS, i.e., $s(\mathcal{T}_2) < s(\mathcal{T})$. Let p_c be the unlearning probability of the client c. Then, Huffman coding with n symbols representing clients and weights $\{p_c\}_{c=1}^{n}$ gives the optimal binary influence tree $\mathcal{T}_{\text{Huffman}}$ such that $s(\mathcal{T}_{\text{Huffman}}) \leq s(\mathcal{T}_2)$ for any binary influence tree for the same group of clients.*

The proof is given in Appendix 1. With this result, we can use Huffman coding (Huffman, 1952) to construct an influence tree that has the lowest IDS. In some real-life applications, the client's unlearning probability can be unknown. In such cases, we can assume that each client is equally likely to be unlearned, hence having the same unlearning probability. A client (on the leaf node) influences a sub-FL model if there is a path from the model to the leaf node representing that client. We now present our algorithm, MMT, which uses both local and sub-FL models to get a better initialization for the global model after unlearning. MMT is designed to easily integrate into various stages of any existing FL algorithm (as depicted in Fig. 1), as outlined below.

- **Initialization.** The central server starts the standard FL training process (see Sect. 3) by randomly initializing the global model. This randomly initialized global model is then shared with all clients. Each client updates the global model using its local training data and then shares the model update (updated model or gradients) with the server. Compared to the standard initialization in any FL training process, each client makes a copy of the locally updated global model[1] (i.e., local model). As the initial global model is randomly initialized, these local models are, by design, isolated from the influence of other clients'

[1] In the first communication round, the locally updated global model is the same as the model that is a copy of the initial global model and trained on the client's training data.

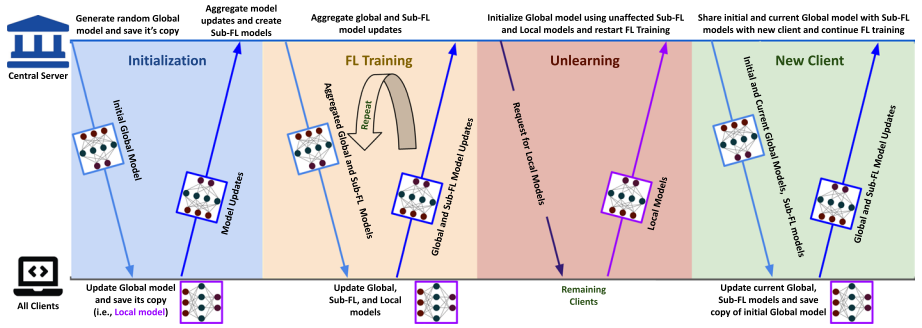


Fig. 1 Multi-Models Training (MMT) in the different stages of any federated learning algorithm. Global and sub-FL training proceed in parallel but may not be synchronized across rounds.

training data. MMT then initializes the sub-FL models using the model updates of clients corresponding to the sub-FL models.

- **FL training.** After receiving the model updates, the central server aggregates them to get the aggregated global and sub-FL models, and then each client receives these updated global and its sub-FL models from the central server and then trains them using its training data. After updating these models, each client shares the global and its sub-FL model updates with the central server. Apart from this, each client also updates their local model. MMT maintains global, sub-FL, and local models concurrently, and their updates do not need to be synchronized with the training rounds of the global FL model. By eliminating overlapping influence, MMT incurs at most a *logarithmic multiplicative overhead* in training cost relative to RFS. This overhead is because sub-FL models at the same depth in the tree have disjoint datasets, so their combined training cost is roughly equivalent to training the global FL model (assuming unchanged hyperparameters). Additionally, the depth of a balanced binary tree scales sub-linearly ($\lceil \log_2 n \rceil$, or more generally $\lceil \log_b n \rceil$ for branching factor b) with the number of leaf nodes, provided the unlearning probability across clients remains approximately uniform (see Sects. 4.2 and 5.3 branching factor discussion). Unlike other exact FU algorithms (Tao et al., 2024; Xiong et al., 2023) with linear memory costs from storing model checkpoints, our approach does not require storing extra checkpoints during training.
- **Unlearning.** To unlearn a client, the server first discards the current global and affected sub-FL models, including all associated optimizer/training state (e.g., momentum/Adam moments, scheduler state, batch-norm running statistics, and any server-side optimizer state), to ***ensure the complete removal of any residual influence from the unlearned client, thereby providing exact unlearning guarantees*** (see formal proof in Appendix 2). The server then requests local model copies from clients not involved in any remaining sub-FL models. Once the server receives all requested local models, it aggregates them with the most influential unaffected sub-FL models (excluding affected descendants) to initialize the new global and sub-FL models, following the FL algorithm. After removing the affected sub-FL models, the remaining influence tree may no longer minimize IDS for the remaining clients. This leaves two options: (i) build a new influence tree while reusing existing sub-FL models as much as possible, or (ii) continue using the current tree, which may be suboptimal but retains previously trained sub-FL models. The server then restarts

FL training with newly initialized global and sub-FL (if any) models, with freshly initialized optimizer/training state, fully free from the unlearned client's influence.

- **New client.** Adding a new client during ongoing FL can degrade the current influence tree compared to rebuilding it with the new client from the start. When a client requests to join, the server waits for the current round to finish. Once complete, the server can either build a new influence tree reusing existing sub-FL models or continue with the current tree, preserving previously trained sub-FL models and adding new ones as needed. The central server begins FL training with the new client by sharing the current global and sub-FL models, which the client updates using its data and returns the updates to the central server. It also provides a randomly initialized global model that the client updates and adopts as its local model for future rounds.

Bi-Models Training (BMT) *is a special case of* MMT without sub-FL models. BMT is more resource-efficient but has lower post-unlearning performance, though still better than RFS. Overall, MMT initializes the global model with joint influence from multiple clients, outperforming BMT and RFS in post-unlearning performance, as shown in Sect. 5. This improved performance comes at the higher pre-training cost due to training both local and sub-FL models, a trade-off justified in applications requiring exact unlearning and fast, high-quality model deployment. More details and illustrations related to MMT and BMT are provided in supplementary material.

Our next result provides exact unlearning guarantees for BMT and MMT.

Theorem 2 *BMT and MMT achieve exact unlearning.*

Proof sketch. To unlearn client u, both BMT and MMT remove all models whose influence sets $\mathcal{I}(\cdot)$ include updates from client u. FL training is then re-initialized from the remaining models, without ever accessing D_u. As a result, client u has no influence on the post-unlearning models. The complete proof is given in Appendix 2.

4.2 Storage and Computational Complexity

In this section, we analyze the storage and computational complexity for RFS, BMT, and MMT, where M denotes the storage required for a single model, C is the baseline training cost of RFS, and n is the number of participating clients. Storage complexity is as follows: RFS requires M storage on the server and M per client; BMT doubles the per-client storage to $2M$ for an additional local model, while server still requires M; and MMT requires server to store $(n-1)M$ models for the internal nodes of a balanced influence tree with n leaf nodes. Each client in MMT, participating in a maximum of $\lceil \log_2 n \rceil$ models and maintaining a local model, uses a maximum of $(\lceil \log_2 n \rceil + 1)M$. While for MMT the server stores $O(n)$ sub-FL models, cross-silo settings usually contain fewer than a hundred parties, so the memory cost is typically in the range of Gigabytes (Table 6), which is small relative to modern server hardware. Moreover, applying memory-efficient model training techniques such as LoRA/QLoRA can further reduce the requirement by only storing model adapters (Babakniya et al., 2023; Dettmers et al., 2023; Hu et al., 2022; Wang et al., 2024; Yi et al., 2023).

In terms of computational complexity (assuming zero communication overhead), RFS has the baseline cost of C, corresponding to one full round of training across all clients; BMT

approximately doubles this cost to $2C$, as each client must update both the global model and an additional isolated model; and MMT organizes clients into a balanced influence tree, the computational cost increases with the tree depth. Specifically, each of the $\lceil \log_2 n \rceil$ internal levels processes all n clients once, and each client also trains its isolated model. This results in a maximum total computational cost of $(\lceil \log_2 n \rceil + 1)C$. Table 1 summarizes storage and computational complexity. The empirical results in Sect. 5.2 closely align, with storage usage matching exactly and compute usage within 8%. Moreover, applying memory-efficient training techniques such as LoRA or QLoRA can further reduce memory requirements by storing only lightweight model adapters instead of the full model parameters; see supplementary material for measured overhead reductions.

5 Experiments

In this section, we first describe our experimental setup.

Datasets & Models. We conduct experiments across four vision tasks: MNIST (LeCun et al., 1998) and FashionMNIST (FMNIST) (Xiao et al., 2017) using MLPs with 30 and 80 hidden units, respectively; CIFAR-10 (Krizhevsky & Hinton, 2009) using a 2-layer CNN with 5×5 convolutional layers, 2×2 max pooling, two fully connected layers (32 hidden units) and ReLU; and CIFAR-100 (Krizhevsky & Hinton, 2009) using VGG-16 model (Simonyan & Zisserman, 2015). We also finetune a pretrained GPT-2 (Radford et al., 2019) and Llama-3.2-3B model (Grattafiori et al., 2024) for next-token prediction on two text datasets: language identification (LI)[2] and multilingual sentiment analysis (MSA).[3]

FL Setup & Training. Our experiments focus on non-IID data silos. For simplicity, we assume the number of clients equals the number of classes in our datasets. Due to high training cost, we only consider the top-8 classes with the most training samples on LI and MSA. To simulate non-IID data, we assign client i with most samples from class i and a few from each of the remaining classes. We denote ρ as the ratio between the majority and minority classes. Clients' datasets have the same sizes of $\mathcal{D}_c^{train}$ for training and $\mathcal{D}_c^{test}$ for testing. While this setting only amplifies label skewness and keeps dataset sizes fixed, we also provide results for a Dirichlet-based non-IID setting where both label proportions and the dataset sizes differ across clients in Fig. 5. The dataset configurations and model architectures are summarized in Table 2. All experiments are conducted on NVIDIA L40 (48 GB) and H100 (80 GB) GPUs. For FL training, we adopt a popular FL algorithm FedAvg (McMahan et al., 2017). The training hyperparameters (e.g., number of communication rounds, local epochs, batch size, etc.) are detailed in Table 3. Additionally, we finetune the pretrained

Table 1 Storage and computational resource requirements

Algorithms	Storage (#models)		Total compute
	Server	Per-client	
RFS	M	M	C
BMT	M	$2M$	$2C$
MMT (balanced)	$(n-1)M$	$(\lceil \log_2 n \rceil + 1)M$	$(\lceil \log_2 n \rceil + 1)C$

[2] https://huggingface.co/datasets/papluca/language-identification

[3] https://huggingface.co/datasets/tyqiangz/multilingual-sentiments

Table 2 Summary of the datasets and models

Datasets	#classes	ρ	$\mathcal{D}_c^{train}$ size	$\mathcal{D}_c^{test}$ size	Models
MNIST	10	0.02	200	200	MLP
FMNIST	10	0.02	200	200	MLP
CIFAR-10	10	0.2	1000	300	2-layer CNN
CIFAR-100	100	0.1	400	100	VGG-16
LI	top-8	$\sim 0^a$	200	100	GPT-2 and Llama-3.2+LoRA
MSA	top-8	$\sim 0^a$	200	100	GPT-2 and Llama-3.2+LoRA

[a]Means each client only has samples from 1 class

Table 3 Hyperparameters used for local training

Tasks	#rounds	#epochs	BS	Optimizer	LR	Decay	Clipping
MNIST	100	1	20	SGD	0.01	0.1	10
FMNIST	100	1	20	SGD	0.01	0.1	10
CIFAR-10	300	1	64	AdamW	0.01	0.1	10
CIFAR-100	100	10	64	SGD	0.005	10^{-5}	–
GPT-2 LI	50	1	8	Adam	10^{-6}	0.01	10
GPT-2 MSA	80	1	8	Adam	10^{-6}	0.01	10
Llama-3.2 LI	30	1	8	Adam	10^{-7}	0.01	1
Llama-3.2 MSA	30	1	8	Adam	10^{-7}	0.01	1

BS Batch size, *LR* learning rate

Llama-3.2 model using LoRA adapters with a rank of $r = 16$, scaling factor $\alpha = 32$, and dropout rate = 0.05.

Metrics. We report accuracy on a test set aggregating from all clients' test sets, including those that would join/leave the collaboration at later rounds. Additionally, we conduct membership inference attacks (MIA) (Mattern et al., 2023; Shokri et al., 2017; Zarifzadeh et al., 2024) to evaluate whether the membership status (i.e., being included during FL training) of unlearned clients can be inferred from the global model's output post-unlearning. An exact FU algorithm should have an MIA AUC close to 0.5, showing that the global model retains no distinguishing information about the unlearned clients' data beyond random guessing.

Baselines. We evaluate BMT and MMT against four exact FU baselines:

(1) **Retraining from Scratch** (RFS): restarts FL training using the remaining clients;
(2) **FedCIO** (Qiu et al., 2023): clients are clustered by cosine similarity of their local model parameters and unlearned by retraining the affected cluster;
(3) **Exact-Fun** (Xiong et al., 2023): stores all local models and a quantized global model per round. Upon unlearning, the global model is recomputed using the remaining clients and compared with the stored quantized model at each round to initiate retraining[4];

[4]Given $100 - 1000$ rounds in typical FL training, storing auxiliary models per round makes Exact-Fun extremely memory-inefficient, and even **infeasible in our Llama-3.2 experiments**. In contrast, the storage cost of BMT and MMT depends only on the number of clients, making them more memory-efficient in cross-silo settings with 10–100 clients.

(4) **FATS** (Tao et al., 2024): randomly samples a subset of clients per round and retrains from the round in which the unlearned client first participates. We note that all baselines except FATS utilize the full set of clients during training.

5.1 Experimental Results

Sequential Unlearning. Figure 2 shows the test accuracy of the global model after sequentially unlearning clients $\{0, 2\}$[5] in vision tasks and client 0 in language tasks, with each unlearning request marked by. ▲ Both methods consistently outperform other baselines and even surpass RFS by a large margin on CIFAR-10. Among the two methods, MMT often achieves faster convergence than BMT post-unlearning. Additionally, Table 4 shows that both methods significantly reduce the number of communication rounds to reach the post-unlearning accuracy thresholds compared to RFS across all tasks. These results highlight the scalability and effectiveness of our approaches across different modalities and model architectures/sizes.

MIA results. Table 5 reports the MIA AUC on the global model *before* (No Unlearning) and *after* unlearning. On most tasks BMT and MMT successfully reduce the MIA success rate post-unlearning, and across all settings they closely match the gold-standard RFS performance. These results highlight that our approaches have effectively removed the influence of unlearned clients from the global model.

Continual Learning & Unlearning. Figure 3 shows the test accuracy of the global model in a dynamic environment where clients join (marked by ▲) and leave (marked by ▲) the collaboration over time. Overall, these results highlight the practical utility of our approaches in supporting simultaneous learning and unlearning within real-world environments.

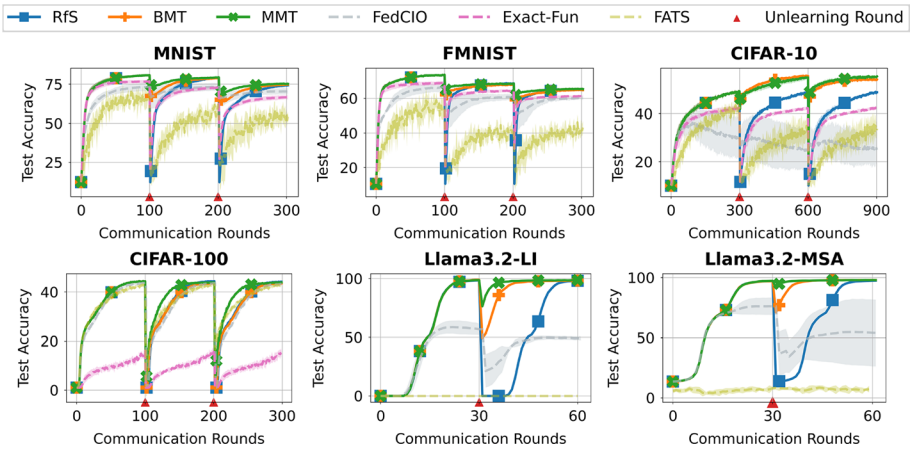


Fig. 2 Global model performance in the sequential unlearning setting across different datasets. Results are averaged over 5 random seeds, with standard deviations shown as shaded regions

[5] This order is chosen to MMT's disadvantage as the global model cannot be replaced by any single sub-FL model and must perform aggregation from multiple sub-FL models.

Table 4 Number of communication rounds to reach post-unlearning accuracy thresholds for the best-performing methods

Tasks	Acc. (%)	RfS	BMT (Δ)	MMT (Δ)
MNIST	75	51	34 (-17)	**16** (-35)
FMNIST	65	26	16 (-10)	**3** (-23)
CIFAR-10	45	162	**3** (-159)	**3** (-159)
CIFAR-100	40	54	54 (+0)	**37** (-17)
GPT-2 LI	70	16	11 (-5)	**6** (-10)
GPT-2 MSA	65	49	22 (-27)	**7** (-42)
Llama-3.2 LI	85	21	6 (-15)	**2** (-19)
Llama-3.2 MSA	85	19	4 (-15)	**1** (-18)

(Δ) shows the difference relative to RfS. MMT and BMT significantly reduced the required rounds compared to RfS

Table 5 MIA AUC on the global model

Tasks	No Unl.	RfS	BMT	MMT
MNIST	0.609	0.520	0.522	0.532
FMNIST	0.508	0.492	0.490	0.490
CIFAR-10	0.489	0.496	0.485	0.489
CIFAR-100	0.911	0.521	0.542	0.517
GPT-2 LI	0.724	0.632	0.618	0.632
GPT-2 MSA	0.715	0.480	0.523	0.527
Llama-3.2 LI	0.553	0.612	0.612	0.612
Llama-3.2 MSA	0.639	0.485	0.485	0.485

BMT and MMT achieve comparable results to RfS across all settings, indicating the influence of unlearned clients has been effectively removed from the global model

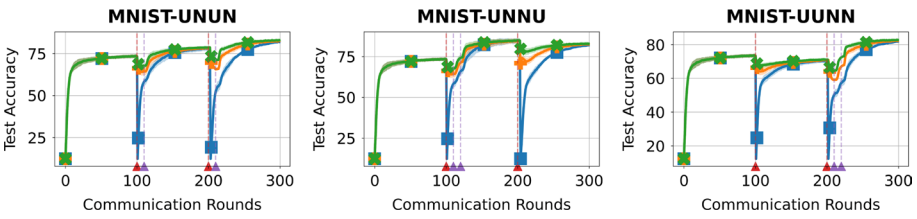


Fig. 3 Global model performance on MNIST under varying learning and unlearning orderings (**N**: New Client Arrival, **U**: Unlearning). Results are averaged across 3 random seeds

5.2 Efficiency

Table 6 presents the peak GPU memory usage (in GB) and total compute cost (in GFLOPs) of our algorithms, which corroborates our analysis in Sect. 4.2. We note that although both algorithms add storage and compute overheads, *storage is no longer the primary bottleneck when training large models* as disk space is relatively inexpensive. Meanwhile, retraining time directly translates to service downtime. Additionally, when compared to existing algorithms, Exact-Fun and FATS require storing model checkpoints per round. Thus, their memory usage grows *linearly* with training duration. In contrast, BMT and MMT scale with the number of clients, which is typically small in cross-silo settings. In time-critical applications involving sensitive, private data, the improved post-unlearning initialization and faster convergence of BMT and MMT in Sect. 5.1 offer a favorable trade-off for their modest overhead.

Deployment feasibility beyond FLOPs. In large-model deployments, practical bottlenecks often include communication volume and peak memory usage, not only total compute. We therefore evaluate a deployment-oriented instantiation of BMT and MMT using parameter-efficient fine-tuning (QLoRA), where a shared quantized backbone is maintained and only lightweight per-node adapters are trained and communicated. We report end-to-end overheads (communication, storage, peak client/server memory, and total FLOPs) for GPT-2 and Llama-3.2-3B in supplementary material, showing that adapters reduce per-update payload and total communication by orders of magnitude and substantially reduce peak memory, improving practicality in the cross-silo setting.

5.3 Ablation Studies

Label Imbalance Ratio. Figure 4 presents the model performance on MNIST across varying label imbalance ratios ρ, where $\rho = 1.0$ indicates uniform label distribution and $\rho \approx 0$ indicates extreme imbalance between the majority and minority classes. We observe that the gaps among the tested methods become more pronounced with lower ρ (higher imbalance), with MMT converging the fastest, followed by BMT and RFS. We attribute this improved

Table 6 Peak GPU memory (GB) and total compute (GFLOPs)

Tasks	Algo.	Memory (GB)		GFLOPs
		Server	Per-client	
MNIST	RFS	1.56	1.56	36.43
	BMT	1.56	3.12	72.86
	MMT	14.03	7.80	161.07
FMNIST	RFS	1.56	1.56	97.15
	BMT	1.56	3.12	194.31
	MMT	14.05	7.81	429.54
CIFAR-10	RFS	2.15	2.15	1.84×10^5
	BMT	2.15	4.30	3.68×10^5
	MMT	19.35	10.75	8.13×10^5
CIFAR-100	RFS	2.75	2.75	5.59×10^7
	BMT	2.75	5.49	1.12×10^8
	MMT	271.90	21.97	4.13×10^8

Memory is reported separately for the server and each client

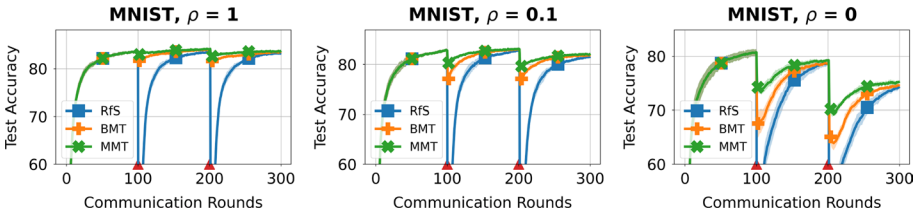


Fig. 4 Effects of label imbalance ratio. While BMT and MMT show comparable performance under uniform label distributions ($\rho = 1$), MMT shows the fastest convergence rate as label distributions become increasingly imbalanced, followed by BMT and RFS

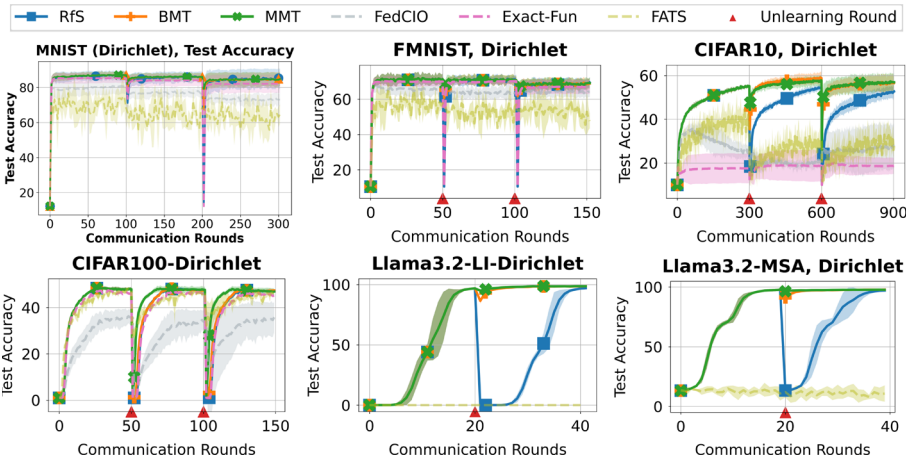
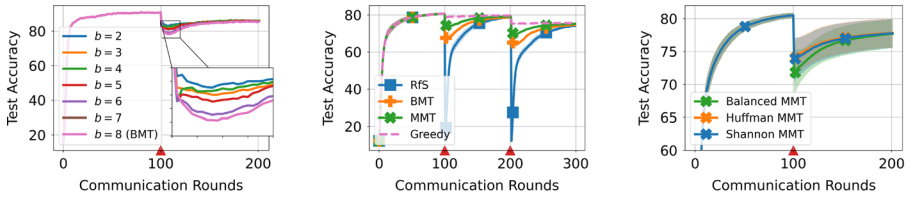


Fig. 5 Global model performance in Dirichlet-based non-IID setting (3 random seeds)

post-unlearning performance to the binary structure in MMT, which preserves the majority of cross-client interactions. These results further demonstrate that our approaches are well-suited for real-world deployments where non-IID data distributions across clients are often the norm.

Dirichlet-based non-IID setting. Figure 5 shows the global model performance when the sample sizes for each class per client follow a Dirichlet distribution, with $\alpha = 0.5$ for vision tasks and $\alpha = 0.1$ for language tasks. Our results show that MMT and BMT continue to achieve better post-unlearning performance than RFS and other baselines, especially on more complex tasks like CIFAR-10 and CIFAR-100.

Branching factor. Figure 6(a) shows how the branching factor $b \in [1, n]$ of the influence tree affects performance, where $b = 1$ corresponds to RFS (hence omitted) and $b = n$ recovers BMT (here $n = 8$, as our ablations use 8 classes). Varying b provides an explicit *compute-performance trade-off*: larger b yields a shallower tree (smaller depth) and therefore fewer sub-FL models per client to update, reducing the additional training overhead. Conversely, a smaller b yields deeper trees and higher pre-training cost. On the performance side, a smaller b tends to yield higher test accuracy because each sub-FL model is trained on a larger subset of clients (roughly $\frac{n}{b}$ clients at the first level in a balanced tree), which



(a) MMT's branching factor (b) Known unlearning order (c) Non-uniform unlearning prob.

Fig. 6 Global model performance on MNIST with (**a**) different branching factor b, (**b**) known unlearning order, and (**c**) non-uniform unlearning probabilities (5 random seeds)

better captures clients' cross-influence on the global model and provides a stronger post-unlearning initialization. In contrast, the endpoint $b = 8$ (BMT) is the most compute-efficient special case but does not maintain intermediate sub-FL models that capture joint influence, which can lead to weaker post-unlearning initialization than a smaller b. Overall, the default binary structure ($b = 2$) offers the best trade-off in our settings.

Known unlearning order. If the unlearning order is known in advance, e.g., clients can subscribe to the service for a fixed duration and then leave upon expiration, an optimal MMT structure can be constructed by placing soon-to-expire clients near the root and greedily building the tree. Figure 6(b) shows that greedy MMT outperforms the balanced MMT, which makes it preferable when the order is known.

Non-uniform unlearning probabilities. Figure 6(c) shows the model performance when the unlearned client is sampled 100 times with pre-defined non-uniform unlearning probabilities under different tree construction algorithms. MMT structures based on Shannon-Fano coding and Huffman coding visibly outperform the default balanced MMT, with Huffman-mmt achieving slightly better results than the Shannon-Fano counterpart. These results align with the classical information theory results that Huffman coding is optimal, and at least as good as Shannon-Fano coding for prefix-free code (Cover & Thomas, 2005).

6 Conclusion

This work proposes two algorithms, BMT and MMT, for exact federated unlearning in the ongoing federated learning collaboration. Our algorithms ensure the complete removal of an unlearned client's data while having better performance post-unlearning with the remaining clients than retraining from scratch. Our algorithms are particularly useful in practical scenarios where model updates in collaborative environments cannot afford long delays and there is minimal tolerance for interruptions. Our extensive experimental results demonstrate the effectiveness of the proposed algorithms. A few promising directions for future work include developing principled algorithms for designing influence trees under resource constraints (e.g., a limited number of trainable sub-FL models), as well as exploring how to adapt the tree after unlearning or client addition in order to minimize IDS while maximizing reuse of existing sub-FL models. Another important direction is integrating BMT and MMT into existing federated learning frameworks to improve their usability, reproducibility, and ease of adoption in practical deployments. Finally, BMT and MMT could be adapted to cross-device settings by selecting a larger branching factor b and using parameter-efficient adapters to reduce client-side overhead.

Appendix 1: Proof of Theorem 1

Proof We first define the k-split influence node, which is a node in an influence tree with $k > 2$ leaf nodes. We now consider the influence tree $\mathcal{T}$, where a k-split influence node only has leaf nodes as children. We denote this node as d, and the set of its leaf nodes is denoted by $\mathcal{C}$. We now follow the following procedure. First, we remove the edge between the node d and any two of its leaf nodes (siblings), denoted by i and j. We create a sub-FL model with these two nodes removed and then add the node for this sub-FL model as a child to the node of d. We denote the resulting tree as $\mathcal{T}'$. Let $f(\mathcal{T}, c)$ represent the number of sub-FL and local models that are aggregated to get the initial global model after unlearning client c in the given influence tree $\mathcal{T}$.

Note that $f(\mathcal{T}', c) = f(\mathcal{T}, c) - 1$ for $c \in \mathcal{C} \setminus \{i, j\}$ as one less leaf node to aggregate due to sub-FL model for $\{i, j\}$ leaf nodes, and $f(\mathcal{T}', c) = f(\mathcal{T}, c)$ for $c \in \{i, j\}$ as sub-FL model is no longer useful due to influence of leaf node i or j. With this, we have the following IDS due to the node d:

$$\begin{aligned} s_d(\mathcal{T}) &= \sum_{c\in\mathcal{C}} p_c f(\mathcal{T}, c) = \sum_{c\in\mathcal{C}\setminus\{i,j\}} p_c(f(\mathcal{T}', c) + 1) + \sum_{c\in\{i,j\}} p_c f(\mathcal{T}', c) \\ &= \sum_{c\in\mathcal{C}\setminus\{i,j\}} p_c + \sum_{c\in\mathcal{C}} p_c f(\mathcal{T}', c) \qquad \text{(node d had k leaf nodes)} \\ &= \sum_{c\in\mathcal{C}\setminus\{i,j\}} p_c + s_d(\mathcal{T}') \\ \implies s_d(\mathcal{T}) &> s_d(\mathcal{T}'). \qquad \text{(since } k > 2 \Rightarrow \mathcal{C} \setminus \{i, j\} \neq \emptyset,\ p_c > 0) \end{aligned} \tag{2}$$

Iteratively apply the same procedure on the rest of the child nodes until every node only has two children. After this, we obtain a binary tree. Since each operation strictly reduces IDS, $s_d(\mathcal{T}_2) < s_d(\mathcal{T})$. When the original tree already has some child nodes L that already belong to a binary subtree $\mathcal{T}_L$, we treat this subtree as a single child node c'_k and apply the aforementioned operations on the child nodes that do not yet belong to a binary subtree. If all the child nodes belong to a binary subtree, we check them from bottom up to find the largest binary subtrees and treat them as a single child node to apply the aforementioned operations. Following this procedure, we can transform any arbitrary tree into a binary tree. In general,

$$f(\mathcal{T}', l) = f(\mathcal{T}_L, l) + f(\mathcal{T}', c') = f(\mathcal{T}_L, l) + f(\mathcal{T}, c') - 1 = f(\mathcal{T}, l) - 1$$

for $c' \in \mathcal{C} \setminus \{i, j\},\ l \in L$ and $f(\mathcal{T}', l) = f(\mathcal{T}_L, l) + f(\mathcal{T}', c') = f(\mathcal{T}_L, l) + f(\mathcal{T}, c') = f(\mathcal{T}, l)$ for $c' \in \{i, j\}, l \in L$. Notice that $f(\mathcal{T}', l) = f(\mathcal{T}_L, l) + f(\mathcal{T}', c')$ always holds. Therefore, the inequality in Eq. 2 generalizes for any tree structure with the generalized operation. After applying this procedure on any arbitrary tree $\mathcal{T}$ with at least one k-split influence node, the resulting binary tree $\mathcal{T}_2$ always has a strictly smaller value of IDS, i.e., $s(\mathcal{T}_2) < s(\mathcal{T})$. Now we will prove the second part of the theorem. Assume $N_{\mathcal{F}}$ is the number of non-root nodes, q_d is the probability of reaching a non-root node d starting from the root node, and N_d^s is the number of siblings of a non-root node d. Note that for a node d, $q_d = \sum_{c\in\mathcal{C}_d} p_c$ where $\mathcal{C}_d$ is the set of all the client nodes (i.e., leaf nodes) that are descendants of node d and p_c is the probability of unlearning of the c-th descendant. Given an influence binary tree $\mathcal{T}_2$ having N clients with known unlearning probability of each client, the IDS is given as follows:

$$s(\mathcal{T}) = \sum_{c=1}^{N} p_c f(\mathcal{T}, c) = \sum_{d=1}^{N_r} q_d \cdot N_d^s. \tag{3}$$

To rewrite $\sum_{c=1}^{N} p_c f(\mathcal{T}, c)$, observe that unlearning a leaf c requires aggregating exactly the siblings of the nodes along the root-to-c path, so $f(\mathcal{T}, c) = \sum_{d \in \text{anc}(c)} N_d^s$, where anc$(c)$ denotes the set of non-root ancestors of leaf c. Substituting this and swapping the order of summation (each non-root node d is an ancestor of exactly the leaves in $\mathcal{C}_d$) gives

$$\sum_{c=1}^{N} p_c f(\mathcal{T}, c) = \sum_{c=1}^{N} p_c \sum_{d \in \text{anc}(c)} N_d^s = \sum_{d=1}^{N_r} N_d^s \sum_{c \in \mathcal{C}_d} p_c = \sum_{d=1}^{N_r} q_d \cdot N_d^s.$$

Since each node of a binary tree has exactly one sibling, $N_d^s = 1$ for all d, and hence

$$\sum_{d=1}^{N_r} q_d \cdot N_d^s = \sum_{d=1}^{N_r} q_d = \sum_{d=1}^{N_r} \sum_{c \in \mathcal{C}_d} p_c = \sum_{c=1}^{n} p_c \cdot l_c,$$

where n is the number of leaf nodes and l_c is the depth of the c-th leaf node (i.e., its number of non-root ancestors). The last equality holds because each leaf c belongs to $\mathcal{C}_d$ for precisely its l_c non-root ancestors d, so p_c is counted exactly l_c times in $\sum_{d=1}^{N_r} \sum_{c \in \mathcal{C}_d} p_c$. Finally, $\sum_{c=1}^{N} p_c \cdot l_c$ is exactly the expected codeword length when the same binary tree is interpreted as a binary prefix-free code with symbol probabilities $\{p_c\}_{c=1}^{N}$.

Since Huffman coding minimizes the expected codeword length, $s(\mathcal{T}_{\text{Huffman}}) \leq s(\mathcal{T}_2)$, where $\mathcal{T}_{\text{Huffman}}$ is the influence tree constructed via Huffman coding and $\mathcal{T}_2$ is any binary influence tree for the same probabilities $\{p_c\}_{c=1}^{N}$. □

Appendix 2: Proof of Exact Unlearning Guarantees

We need the following definitions and results, which will be used to prove Theorem 2.

Training model. Let $D = (D_1, \ldots, D_N)$ denote client datasets and let r denote all explicit randomness, independent of D. A training *state* is $m = (\theta, \xi)$, consisting of model parameters and optimizer/training state. Let M denote server-visible metadata and protocol choices; after removing client u, the metadata is denoted M_{-u}.

Definition 4 (*Training primitives*) A *state* is $m = (\theta, \xi)$ with parameters θ and training state ξ. Fix server-visible protocol choices M and explicit randomness r, and let ξ_0 denote a canonical fresh training state. Training is modeled as compositions of deterministic primitives:

Init $(r) = (\theta_0(r), \xi_0)$: initialize the random model parameters and fresh training states.
Copy $(m) = m$: make an identical copy of the current model state.
ResetOpt $((\theta, \xi), r) = (\theta, \xi_0)$: keep model parameters, clean up optimizer history
TrainOn (m, D_i, r) : the M-specified local training procedure on D_i starting from m, and
AGG $(m_1, \ldots, m_k; M, r)$: the M-specified server aggregation rule.

Only TrainOn needs private client data, and it reads only the dataset passed to it. Conditioned on (r, M), the sequence of primitive calls and all non-data arguments (including scheduling, aggregation rules, and hyperparameters) do not depend on any D_i except through the explicit appearance of D_i as an argument to TrainOn. After unlearning client u, the orchestration uses only M_{-u}.

Definition 5 (*Influence set*) For any state m produced by the training computation, define its *influence set* $\mathcal{I}(m) \subseteq \mathcal{C}$ as the set of clients whose datasets may influence m. Formally, $\mathcal{I}(m)$ is the smallest set satisfying:

(i) $\mathcal{I}(\text{Init}\ (r)) = \emptyset$;
(ii) data-free primitives (Copy, ResetOpt) preserve influence sets;
(iii) $\mathcal{I}(\text{AGG}\ (m_1, \ldots, m_k)) = \bigcup_{j=1}^{k} \mathcal{I}(m_j)$;
(iv) $\mathcal{I}\text{TrainOn}\ (m, D_i, r)) = \mathcal{I}(m) \cup \{i\}$. For a deployed model h_θ extracted from $m = (\theta, \xi)$, define $\mathcal{I}(h_\theta) \doteq \mathcal{I}(m)$.

Lemma 1 *(Dependence only on influence set) Fix (r, M) and assume data-independent orchestration. For any state m, the value of m depends on the datasets only through* $\{D_i\}_{i \in \mathcal{I}(m)}$. *Equivalently, if* $u \notin \mathcal{I}(m)$, *then for any replacement* D'_u *that leaves M unchanged,*

$$m(D_1, \ldots, D_u, \ldots, D_N; r, M) = m(D_1, \ldots, D'_u, \ldots, D_N; r, M).$$

Proof Conditioned on (r, M), the training expression generating m is fixed. We proceed by structural induction over this expression. Init depends only on r.

Data-free primitives are deterministic functions of their inputs and introduce no new data dependence. Agg depends only on its input states and thus on the union of their influence sets. TrainOn accesses only (m, D_i, r) and introduces dependence on D_i. In all cases, m depends only on datasets indexed by $\mathcal{I}(m)$. □

Definition 6 (*Exact federated unlearning*) An unlearning procedure U for client u is *exact* if, for all replacements D'_u that keep metadata unchanged,

$$U(D_1, \ldots, D_u, \ldots, D_N; r, M_{-u}) = U(D_1, \ldots, D'_u, \ldots, D_N; r, M_{-u}).$$

Proof of Theorem 2

This section presents the formal proof of mmt and bmt achieving exact unlearning.

Proof Fix r and M_{-u}. Consider two models differing only in the removed client's dataset, D_u versus D'_u. bmt and mmt discard all stored states m with $u \in \mathcal{I}(m)$ and retain only states $\{m_1, \ldots, m_k\}$ satisfying $u \notin \mathcal{I}(m_j)$. By Lemma 1, each retained state m_j is invariant to replacing D_u by D'_u, so the retained collection is identical in both models.

The post-unlearning initialization

$$m_{\text{post}}^{(0)} = \textsc{ResetOpt}(\text{AGG}(m_1, \dots, m_k; M_{-u}, r), r)$$

is a deterministic function of $(m_1, \dots, m_k, M_{-u}, r)$ and therefore identical in both models. Moreover, since Agg unions influence sets and ResetOpt preserves them,

$$\mathcal{I}\left(m_{\text{post}}^{(0)}\right) = \bigcup_{j=1}^{k} \mathcal{I}(m_j),$$

so it excludes u because every retained m_j excludes u.

Subsequent training invokes TrainOn only on datasets D_i with $i \neq u$. Since TrainOn is the only primitive that can add a client to the influence set, u is never reintroduced. Thus, the final deployed state m_{post} satisfies $u \notin \mathcal{I}(m_{\text{post}})$. Applying Lemma 1, m_{post} is invariant to

replacing D_u by D'_u, which is exactly the definition of exact unlearning. □

Remark 1 (*Exactness vs. retraining-from-scratch*) RfS is also exact under Definition 6 since it never invokes $\text{TrainOn}(\cdot, D_u, \cdot)$ after removing u; BMT and MMT need not match RfS parameters; rather, they satisfy the same invariance notion with a better initialization.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10994-026-07117-8.

Author Contributions Arun Verma proposed the high-level idea of MMT and BMT, contributed significantly to the manuscript writing, and provided feedback to the theoretical results. Ze Yu Zhang implemented the initial version of MMT, BMT and baselines, conducted language tasks experiments and constructed the theory and its proof. Nhung Bui conducted vision-related experiments and ablation studies, and contributed to the writing of this paper. Bryan Kian Hsiang Low supervised and provided computational resources for this project. Bolin Ding supervised Ze Yu Zhang for this project. All authors read and approved the final manuscript.

Funding This research is supported by the National Research Foundation, Singapore under its National Large Language Models Funding Initiative (AISG Award No: AISG-NMLP-2024-001).This research is supported by the National Research Foundation (NRF), Prime Minister's Office, Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) programme. The Mens, Manus, and Machina (M3S) is an interdisciplinary research group (IRG) of the Singapore MIT Alliance for Research and Technology (SMART) centre.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

Aghabagherloo, A., Abadi, A., Sarkar, S., Dasu, V. A., & Preneel, B. (2025). Impact of data duplication on deep neural network-based image classifiers: Robust vs. standard models. In *Proc. IEEE SPW* (Vol. 12, pp. 177–183). https://doi.org/10.1109/SPW67851.2025.00023

Alain, G., Lamb, A., Sankar, C., Courville, A., & Bengio, Y. (2015). Variance reduction in SGD by distributed importance sampling. arXiv:1511.06481

Allamanis, M. (2019). The adverse effects of code duplication in machine learning models of code. In *Proc. ACM SIGPLAN Onward!* (pp. 143–153). https://doi.org/10.1145/3359591.3359735

Bourtoule, L., Chandrasekaran, V., Choquette-Choo, C.A., Jia, H., Travers, A., Zhang, B., Lie, D., & Papernot, N. (2021). Machine unlearning. In *Proc. IEEE S&P* (pp. 141–159). https://doi.org/10.1109/SP40001.2021.00019

Brisimi, T. S., Chen, R., Mela, T., Olshevsky, A., Paschalidis, I. C., & Shi, W. (2018). Federated learning of predictive models from federated electronic health records. *International Journal of Medical Informatics, 112*, 59–67. https://doi.org/10.1016/J.IJMEDINF.2018.01.007

Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review, 60*, 223–311. https://doi.org/10.1137/16M1080173

Babakniya, S., Elkordy, A. R., Ezzeldin, Y. H., Liu, Q., Song, K.-B., El-Khamy, M., & Avestimehr, S. (2023). SLoRA: Federated parameter efficient fine-tuning of language models. In *NeurIPS workshop on federated learning in the age of foundation models*

Byrd, J., & Lipton, Z. (2019). What is the effect of importance weighting in deep learning? In *Proc. ICML* (pp. 872–881).

Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks, 106*, 249–259. https://doi.org/10.1016/J.NEUNET.2018.07.011

Byrd, D., & Polychroniadou, A. (2020). Differentially private secure multi-party computation for federated learning in financial applications. In *Proc. ICAIF* (pp. 1–9). https://doi.org/10.1145/3383455.3422562

CCPA: California Consumer Privacy Act of 2018. (2018). California Civil Code Title 1.81.5.

Cao, X., Jia, J., Zhang, Z., & Gong, N. Z. (2023) FedRecover: Recovering from poisoning attacks in federated learning using historical information. In *Proc. IEEE S&P* (pp. 1366–1383). https://doi.org/10.1109/SP46215.2023.10179336

Cortes, C., Mansour, Y., & Mohri, M. (2010). Learning bounds for importance weighting. In *Proc. NeurIPS* (pp. 442–450).

Cover, T. M., & Thomas, J. A. (2005). *Elements of information theory* (2nd ed.). Wiley. https://doi.org/10.1002/047174882X

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Proc. NeurIPS* (pp. 10088–10115).

Dhade, P., & Shirke, P. (2023). Federated learning for healthcare: A comprehensive review. *Engineering Proceedings, 59*, 230. https://doi.org/10.3390/ENGPROC2023059230

Fan, C., Liu, J., Zhang, Y., Wong, E., Wei, D., & Liu, S. (2024) SalUn: Empowering machine unlearning via gradient-based weight saliency in both image classification and generation. In *Proc. ICLR*.

Fang, M., Cao, X., Jia, J., & Gong, N. (2020). Local model poisoning attacks to byzantine-robust federated learning. In *Proc. SEC* (pp. 1605–1622).

Geiping, J., Bauermeister, H., Dröge, H., & Moeller, M. (2020). Inverting gradients - how easy is it to break privacy in federated learning? In *Proc. NeurIPS* (pp. 16937–16947)

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Ma, Z. (2024). The Llama 3 herd of models. Technical report, Meta. https://ai.meta.com/research/publications/the-llama-3-herd-of-models/

GDPR: General Data Protection Regulation, Article 17. (2016). Right to Erasure ('Right to Be Forgotten'). Official Journal of the European Union (2016). Regulation (EU) 2016/679.

Graves, L., Nagisetty, V., & Ganesh, V. (2021). Amnesiac machine learning. In *Proc. AAAI* (pp. 11516–11524). https://doi.org/10.1609/AAAI.V35I13.17371

Gu, H., Zhu, G., Zhang, J., Zhao, X., Han, Y., Fan, L., & Yang, Q. (2024). Unlearning during learning: An efficient federated machine unlearning method. In *Proc. IJCAI* (pp. 4035–4043). https://doi.org/10.24963/IJCAI.2024/446

Halimi, A., Kadhe, S., Rawat, A., & Baracaldo, N. (2022). Federated unlearning: How to efficiently erase a client in FL? arXiv:2207.05521

He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., Candela J. Q. (2014). Practical lessons from predicting clicks on ads at Facebook. In *Proc. ADKDD* (pp. 1–9). https://doi.org/10.1145/2648584.2648589

Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., Eichner, H., Kiddon, C., & Ramage, D. (2018). Federated learning for mobile keyboard prediction arXiv:1811.03604.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *Proc. ICLR*.

A Huffman, D. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, *40*, 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

Jin, X., Chen, P.-Y., Hsu, C.-Y., Yu, C.-M., & Chen, T. (2021). CAFE: Catastrophic data leakage in vertical federated learning. In *Proc. NeurIPS* (pp. 994–1006).

Khalil, Y.H., Brunswic, L., Lamghari, S., Li, X., Beitollahi, M., & Chen, X. (2025). NoT: Federated unlearning via weight negation. In *Proc. CVPR* (pp. 25759–25769). https://doi.org/10.1109/CVPR52734.2025.02399

Krizhevsky, A., & Hinton, G. (2009). *Learning multiple layers of features from tiny images*. Technical report. University of Toronto. https://doi.org/10.24432/C5889J

Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., D'Oliveira, R. G. L., Eichner, H., El Rouayheb, S., Evans, D., Gardner, J., Garrett, Z., Gascón, A., Ghazi, B., Gibbons, P. B., ... Zhao, S.(2021). Advances and open problems in federated learning. *Foundations and Trends in Machine Learning, 14*, 1–210. https://doi.org/10.1561/2200000083

Lamri, H., Alam, M., Jiang, H., & Maniatakos, M. (2026). DRAUN: An algorithm-agnostic data reconstruction attack on federated unlearning systems. In *Proc. AISTATS*.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. In *Proceedings of the IEEE, 86*, 2278–2324. https://doi.org/10.1109/5.726791

Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. In *Proc. MLSys* (pp. 429–450).

Lim, W. Y. B., Luong, N. C., Hoang, D. T., Jiao, Y., Liang, Y.-C., Yang, Q., Niyato, D., & Miao, C. (2020). Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials,22*, 2031–2063. https://doi.org/10.1109/COMST.2020.2986024

Liu, G., Ma, X., Yang, Y., Wang, C., Liu, J. (2021). FedEraser: Enabling efficient client-level data removal from federated learning models. In *Proc. IWQOS* (pp. 1–10). https://doi.org/10.1109/IWQOS52092.2021.9521274

Liu, N., & Ren, S. (2025). Production disruption in supply chain systems: Impacts on consumers, supply chain agents and the society. *Annals of Operations Research*, *344*, 965–988. https://doi.org/10.1007/S10479-023-05782-9

Liu, Z., Jiang, Y., Shen, J., Peng, M., Lam, K.-Y., Yuan, X., & Liu, X. (2024). A survey on federated unlearning: Challenges, methods, and future directions. *ACM Computing Surveys, 57*, 1–38. https://doi.org/10.1145/3679014

Long, G., Tan, Y., Jiang, J., & Zhang, C.: Federated learning for open banking. In *Federated learning: Privacy and incentive* (pp. 240–254). Springer. https://doi.org/10.1007/978-3-030-63076-8_17

Mattern, J., Mireshghallah, F., Jin, Z., Schölkopf, B., Sachan, M., & Berg-Kirkpatrick, T. (2023). Membership inference attacks against language models via neighbourhood comparison. In *Findings of the Association for Computational Linguistics: ACL 2023* (pp. 11330–11343). https://doi.org/10.18653/V1/2023.FINDINGS-ACL.719

McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proc. AISTATS* (pp. 1273–1282)

Nasr, M., Shokri, R., & Houmansadr, A. (2019). Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *Proc. IEEE S&P* (pp. 739–753). https://doi.org/10.1109/SP.2019.00065

Prayitno, Shyu, C. R., Putra, K. T., Chen, H. C., Tsai, Y. Y., Hossain, K. T., Jiang, W., & Shae, Z. Y. (2021). A systematic review of federated learning in the healthcare area: From the perspective of data properties and applications. *Applied Sciences*, *11*, 11191. https://doi.org/10.3390/APP112311191

Pan, Z., Wang, Z., Li, C., Zheng, K., Wang, B., Tang, X., & Zhao, J.: Federated unlearning with gradient descent and conflict mitigation. In *Proc. AAAI* (pp. 19804–19812). https://doi.org/10.1609/AAAI.V39I19.34181

Qiu, H., Wang, Y., Xu, Y., Cui, L., & Shen, Z. (2023). FedCIO: Efficient exact federated unlearning with clustering, isolation, and one-shot aggregation. In *Proc. IEEE BigData* (pp. 5559–5568). https://doi.org/10.1109/BigData59044.2023.10386788

Rieke, N., Hancox, J., Li, W., Milletari, F., Roth, H. R., Albarqouni, S., Bakas, S., Galtier, M. N., Landman, B. A., Maier-Hein, K., Ourselin, S., Sheller, M., Summers, R. M., Trask, A., Xu, D., Baust, M., Cardoso, M. J. (2020). The future of digital health with federated learning. *npj Digital Medicine*, *3*, 119. https://doi.org/10.1038/S41746-020-00323-1

Rashid, M. R. U., Dasu, V. A., Gu, K., Sultana, N., & Mehnaz, S. (2023). Gradient-free privacy leakage in federated language models through selective weight tampering. arXiv:2310.16152

Romandini, N., Mora, A., Mazzocca, C., Montanari, R., & Bellavista, P. (2025). Federated unlearning: A survey on methods, design guidelines, and evaluation metrics. *IEEE Transactions on Neural Networks and Learning Systems*, *36*, 11697–11717. https://doi.org/10.1109/TNNLS.2024.3478334

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. Technical report. OpenAI.

Rostek, K., Wiśniewski, M., & Skomra, W. (2022). Analysis and evaluation of business continuity measures employed in critical infrastructure during the covid-19 pandemic. *Sustainability*, *14*, 15388. https://doi.org/10.3390/SU142215388

Shokouhi, M. (2013). Learning to personalize query auto-completion. In *Proc. ACM SIGIR* (pp. 103–112). https://doi.org/10.1145/2484028.2484076

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press. https://doi.org/10.1017/CBO9781107298019

Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). Membership inference attacks against machine learning models. In *Proc. IEEE S&P* (pp. 3–18). https://doi.org/10.1109/SP.2017.41

Shi, J., Wan, W., Hu, S., Lu, J.,& Zhang, L. Y. (2022). Challenges and approaches for mitigating byzantine attacks in federated learning. In *Proc. IEEE TrustCom* (pp. 139–146). https://doi.org/10.1109/TRUSTCOM56396.2022.00030

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *Proc. ICLR*.

Tolpegin, V., Truex, S., Gursoy, M. E., & Liu, L. (2020). Data poisoning attacks against federated learning systems. In (*Computer Security—ESORICS* (pp. 480–501). https://doi.org/10.1007/978-3-030-58951-6_24

Tao, Y., Wang, C.-L., Pan, M., Yu, D., Cheng, X., Wang, D. (2024). Communication efficient and provable federated unlearning. *Proceedings of VLDB Endowment, 17*, 1119–1131. https://doi.org/10.14778/3641204.3641220

Vorotilov, V., & Shugaepov, I. (2023). *Scaling the Instagram explore recommendations system*. Meta Engineering Blog. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/

Wu, L., Guo, S., Wang, J., Hong, Z., Zhang, J., & Ding, Y. (2022). Federated unlearning: Guarantee the right of clients to forget. *Ieee Network*, *36*, 129–135. https://doi.org/10.1109/MNET.001.2200198

Wang, J., Guo, S., Xie, X., Qi, H. (2022). Federated unlearning via class-discriminative pruning. In *Proc. WWW* (pp. 622–632). https://doi.org/10.1145/3485447.3512222

Wang, F., Li, B., & Li, B. (2024). Federated unlearning and its privacy threats. *Ieee Network*, *38*, 294–300. https://doi.org/10.1109/MNET.004.2300056

Wallace, W. A., Mendonca, D., Lee, E., Mitchell, J. E., & Chow, J. (2003). Managing disruptions to critical interdependent infrastructures in the context of the 2001 world trade center attack. In *Beyond September 11th: An account of post-disaster research* (pp. 165–198). Natural Hazards Research and Applications Information Center.

Wang, Z., Shen, Z., He, Y., Sun, G., Wang, H., Lyu, L., & Li, A. (2024). FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations. In *Proc. NeurIPS* (pp. 22513–22533).

Wu, C., Zhu, S., & Mitra, P. (2022). Federated unlearning with knowledge distillation. arXiv:2201.09441.

Xiong, Z., Li, W., Li, Y., & Cai, Z. (2023). Exact-fun: An exact and efficient federated unlearning approach. In *Proc. IEEE ICDM* (pp. 1439–1444). https://doi.org/10.1109/ICDM58522.2023.00188

Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747.

Yi, L., Yu, H., Wang, G., Liu, X., & Li, X. (2023). pFedLoRA: Model-heterogeneous personalized federated learning with lora tuning arXiv:2310.13283

Zhu, L., Liu, Z., & Han, S. (2019). Deep leakage from gradients. In *Proc. NeurIPS* (pp. 14774–14784).

Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., & Chandra, V. (2018). Federated learning with non-IID data. arXiv:1806.00582.

Zarifzadeh, S., Liu, P., & Shokri, R. (2024). Low-cost high-power membership inference attacks. In *Proc. ICML* (pp. 58244–58282).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Ze Yu Zhang[1,2] · Nhung Bui[1] · Arun Verma[3] · Bolin Ding[2] · Bryan Kian Hsiang Low[1,3]

✉ Ze Yu Zhang
zhan1130@comp.nus.edu.sg

✉ Nhung Bui
btcnhung@u.nus.edu

✉ Arun Verma
arun.verma@smart.mit.edu

Bolin Ding
bolin.ding@alibaba-inc.com

Bryan Kian Hsiang Low
lowkh@comp.nus.edu.sg

[1] National University of Singapore, Singapore, Singapore

[2] Alibaba Group, Singapore, Singapore

[3] Singapore-MIT Alliance for Research and Technology Centre, Singapore, Singapore